```
def max_point_count
  5
end
```

I do this because then max_point_count has the same lookup semantics as any other instance value, and it often reads better to make a value owned by the instance rather than the class. It's also easier to change if the constant turns out to be less than constant in the future.

Duplication of logic is similar to duplication of fact, but instead of looking for simple values, you're looking for longer structures. This often includes compound Boolean statements used as guards in multiple methods (in the task-manager example, this might be something about whether a task has been completed) or simple calculations that are used in multiple places (such as converting task size to time based on the project's rate of completion).

In this example, the same Boolean test is applied twice and it's easy to imagine it being used many more times:

```
class User
  def maximum_posts
    if status == :trusted then 10 else 5 end
  end

  def urls_in_replies
    if status == :trusted then 3 else 0 end
  end
end
```

One way to mitigate this is to move the duplicated logic into its own method and call the method from each location (in this case, def trusted?). I recommend being aggressive about this—you'll sometimes see advice that you should just notice duplication on the second instance and refactor on the third instance. In my experience, that just means you wind up with twice as much duplication as you should have. (See the next section for other ideas about reused Booleans.)

Keep in mind that not every piece of logic that is spelled the same in Ruby is actually duplication. It's possible for early forms of two pieces of logic to look similar but eventually evolve in separate directions. A great example of this is Rails controller scaffolding. Every RESTful controller starts with the same boilerplate code for the seven RESTful actions. And there have been innumerable attempts to remove that duplication by creating a common abstraction. Most of those attempts eventually wind up in a tangle of special case logic because each controller eventually needs to have different features.